EXPERIMENT – 2: BAYES DECISION THEORY USING SMS SPAM COLLECTION DATASET

AIM

To implement **Bayes Decision Theory** using the **SMS Spam Collection Dataset** and classify SMS messages as **Spam** or **Ham** using the **Multinomial Naive Bayes algorithm**.

THEORY

Bayes Decision Theory is a probabilistic approach used for classification. It is based on **Bayes' Theorem**, which calculates the probability of a class given the observed data.

Naive Bayes assumes that all features are independent of each other, making it simple and efficient for text classification problems such as spam detection.

ALGORITHM

1. Import the required libraries.
2. Load the SMS Spam Collection dataset.
3. Split the dataset into features (messages) and target (spam/ham).
4. Convert text messages into numerical vectors using **CountVectorizer**.
5. Split the dataset into training and testing sets.
6. Train the **Multinomial Naive Bayes** classifier.
7. Predict the labels for the test dataset.
8. Calculate the accuracy of the model.
9. Classify a new SMS message as **Spam** or **Ham**.

PROGRAM

```
import os

import urllib.request

import zipfile

import pandas as pd


from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, classification_report


# Download dataset

if not os.path.exists("SMSSpamCollection"):


    url =
"https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip"


    urllib.request.urlretrieve(url, "smsspamcollection.zip")


    with zipfile.ZipFile("smsspamcollection.zip", "r") as zip_ref:

        zip_ref.extractall()


# Load dataset

df = pd.read_csv(
```

```
    "SMSSpamCollection",
    sep="\t",
    names=["label", "message"]
)

# Features and Labels
X = df["message"]
y = df["label"]

# Convert text to vectors
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train model
model = MultinomialNB()
```

```
model.fit(X_train, y_train)


# Prediction

y_pred = model.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))


print(classification_report(y_test, y_pred))


# Test message

msg = ["Congratulations! You won a free iPhone. Click here to claim."]


msg_vector = vectorizer.transform(msg)


print("Prediction:", model.predict(msg_vector)[0])
```

DATASET

Dataset Name: SMS Spam Collection Dataset

- Source : UCI Machine Learning Repository
- Total Messages : **5574**
- Classes : **Ham** and **Spam**
- Ham Messages : **4827**
- Spam Messages : **747**

OUTPUT

Accuracy: 0.9856502242152466

Classification Report

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	966
spam	0.94	0.95	0.95	149
accuracy			0.99	1115
macro avg	0.97	0.97	0.97	1115
weighted avg	0.99	0.99	0.99	1115

Custom Message Prediction

Message:

Congratulations! You won a free iPhone. Click here to claim.

Prediction:

spam

RESULT

Thus, the **Multinomial Naive Bayes classifier** was successfully trained using the **SMS Spam Collection Dataset**.

The model achieved an accuracy of **98.56%** and correctly classified the given SMS message as **Spam**.